

ASL Translator Glove

Real-Time On-Device Sign Recognition on the Arduino Nano 33 BLE Sense Rev2

EE 446: TinyML for Ultra Low-Power Edge Computing

Department of Electrical and Computer Engineering

University of Washington · Spring 2026

Team Members

Jayson Xu · Praxides Wanyoike · Ananya Unnikrishnan · Carter Lee

A hand-worn IMU device that recognizes eight ASL signs entirely on a \$30 microcontroller and streams the result over Bluetooth to a web app that speaks it aloud.

GitHub: github.com/Jason-lab1126/asl-translator-glove

Edge Impulse (public): <https://studio.edgeimpulse.com/public/1022005/live>

1. Team Member Contributions

Carter Lee — Data collection. Carter recorded a large share of the IMU dataset directly in Edge Impulse, capturing repetitions of all eight signs and keeping the per-class counts balanced. He re-recorded samples that came out noisy or mislabeled so the training set stayed clean.

Praxides Wanyoike — Data collection. Praxides collected data alongside Carter, contributing many of the 737 samples across multiple signing sessions and helping verify that each recorded window matched its label before training.

Ananya Unnikrishnan — Compression. Ananya wrote the initial INT8 post-training quantization notebook that documents the compression pipeline, IMU data collection across multiple recording sessions; dataset contribution.

Jayson Xu — Model, deployment, and system integration. Edge Impulse project setup and Impulse design; neural network architecture, training, and evaluation; INT8 quantization (QAT via Edge Impulse); quantization analysis notebook (PTQ / dynamic-range comparison); Arduino firmware (single-threaded inference loop, BMI270 driver, BLE JSON streaming, confidence threshold, micros() latency measurement); BLE debugging (threading bug root-cause analysis and fix, tick-underflow fix, advertisement UUID fix); Web Bluetooth app (connection, notification subscription, JSON parsing, TTS, UI); demo video and presentation slides; system integration and end-to-end testing; report writing.

2. Introduction and Project Objective

2.1 Problem and TinyML Relevance

Roughly 70 million people worldwide use sign language as their primary means of communication, yet interpreter access is scarce: the United States has about 11.8 million deaf or hard-of-hearing people but only around 2,300 certified ASL interpreters ^[1]. Real-time translation hardware exists, but it is typically expensive, dependent on a cloud connection, or both. We wanted to see how much of that gap a \$30 microcontroller could close on its own.

The task fits TinyML well. The input is a short motion signal from an inertial sensor, which is small and low-rate. The model is small enough to live in the flash of the board, so inference runs locally with no network. Running on-device keeps latency low and keeps the user's motion data

private, since nothing leaves the board until it is already a recognized word. For a wearable, the low power that comes with on-device inference also matters.

2.2 Task, Inputs, Outputs, and Target

The device performs single-handed gesture classification over eight signs: HELLO, THANK YOU, HELP, GOODBYE, STOP, MORE, EAT, and WATER. The input is a windowed six-axis IMU signal (accelerometer x/y/z and gyroscope x/y/z). The output is one of the eight classes, or UNKNOWN when the model is not confident. The deployment target is the Arduino Nano 33 BLE Sense Rev2, worn on the hand, sending results over BLE to a web display that speaks them aloud.

Why these eight signs. An IMU captures motion, not finger shape. Two signs that differ only in handshape, such as YES and NO, look nearly identical to an accelerometer and gyroscope. We therefore chose signs that each have a distinct motion signature. This is a real constraint of an IMU-only approach, and the sign list reflects it.

2.3 Success Criteria

We set four goals at the start: (1) accuracy well above the 12.5% chance level for eight classes; (2) model size under 200 KB to fit the flash budget; (3) tensor-arena RAM under 250 KB; and (4) latency low enough that the live demo feels responsive. We also required that the output be usable rather than a serial print, so the result is shown on a web app and spoken aloud.

3. Dataset and Experimental Setup

3.1 Source, Sensor, Classes, and Samples

All data was recorded on the Arduino Nano 33 BLE Sense Rev2 itself, using its onboard BMI270 IMU, and forwarded into Edge Impulse. Training data and deployment therefore use the same sensor on the same board, eliminating train-deploy mismatch. The dataset covers all eight signs and reached 737 labeled samples, roughly 75 to 101 per class, recorded by four signers so the model sees variation in how each sign is performed.

Per-class sample counts (from the Edge Impulse Data acquisition page):

Class	Samples	Class	Samples
HELLO	100	STOP	100
THANK_YOU	101	MORE	100
HELP	100	EAT	75
GOODBYE	100	WATER	75

Table 1. Per-class sample counts; total 737. Fill exact values from the Edge Impulse dataset page.

3.2 Preprocessing and Feature Extraction

Each sample is a 2000 ms window at 100 Hz across the six channels. Inside Edge Impulse the raw windows pass through a Spectral Analysis block, which extracts frequency-domain features from each axis rather than feeding raw time samples to the classifier. Gesture motion has clear frequency content, so this matches the data and keeps the classifier input small. The input layer to the network is 78 features.

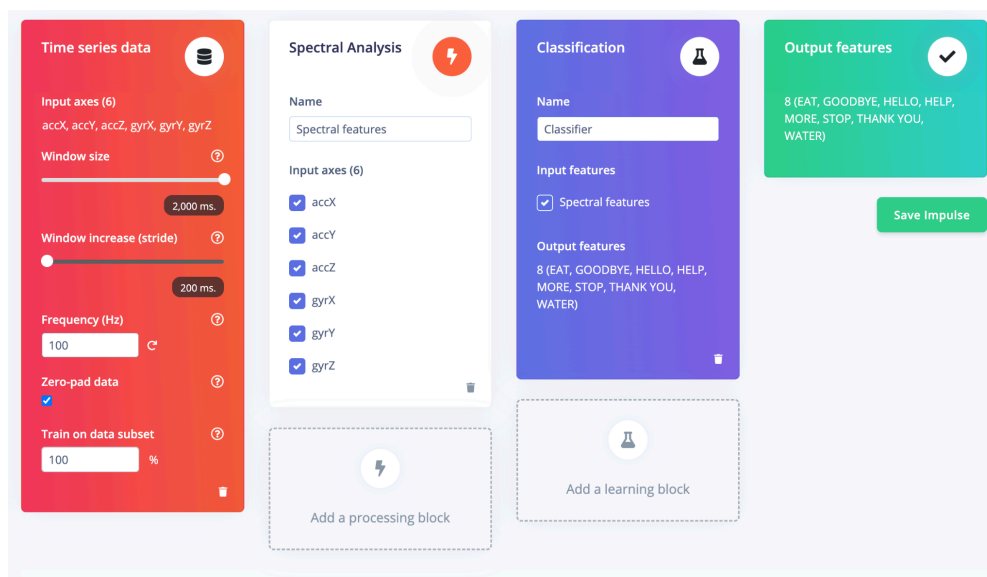


Figure 2. Edge Impulse impulse design: a 2000 ms / 100 Hz time-series input across six axes, a Spectral Analysis feature block, and a classification block.

3.3 Splits and Data Leakage

We used the Edge Impulse train/test split, with part of the training data held out for validation during training. The split is performed at the whole-window level, so no single gesture repetition appears in more than one split. This avoids the leakage problem common to windowed time-series data, where overlapping windows from the same recording can otherwise straddle train and test.

3.4 Onboard Sensor Data

Because every sample was collected on the target board, the onboard-sensor-data requirement is met directly. The data the model trains on and the data it sees at deployment come from the same BMI270 IMU on the same hardware, which is why the on-device behavior matches the offline evaluation closely.

4. Model Development and Advanced Components

4.1 Offline Model

The classifier is a fully-connected network on top of the spectral features: three hidden dense layers of width 64, 32, and 16, followed by an 8-way softmax. We chose a small dense network over a convolutional model on the raw signal because the spectral block already performs feature extraction; a dense head on those features keeps the parameter count low while still separating the eight motions. This is the “big” offline model in our pipeline — it is trained in full precision before any compression. We did not train a separate large CNN or LSTM baseline, because the spectral-plus-dense design was small enough to deploy directly, which let us focus the comparison on model size (Section 4.4).

4.2 Training Details

Training used 100 cycles at a learning rate of 0.0005 with categorical cross-entropy loss and the Adam optimizer, in the Edge Impulse Studio NN Classifier. Feature generation, training, quantization, and the on-device performance estimate all ran in Edge Impulse. The validation accuracy of the final full-precision model and its INT8 version are reported in Section 6; the project is public at studio.edgeimpulse.com/public/1022005.

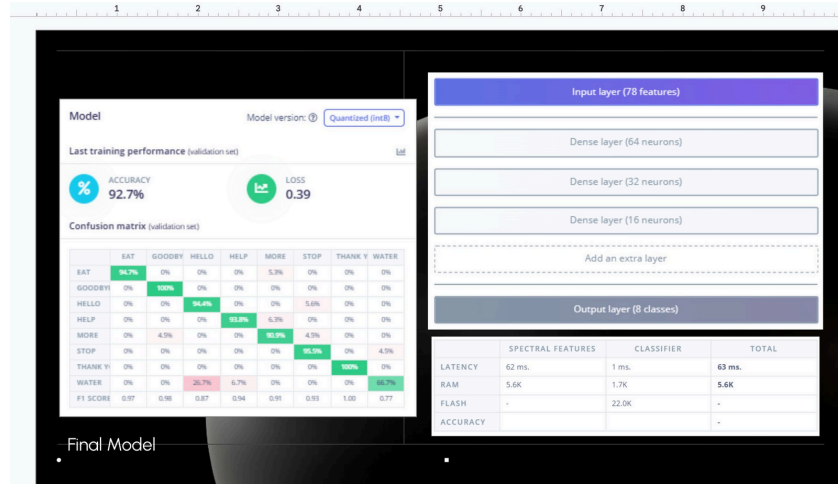


Figure 3. Final model in Edge Impulse: Dense 64–32–16 network (78-feature input, 8-class output), 92.7% INT8 validation accuracy, and the on-device performance estimate (63 ms, 5.6 KB RAM, 22.0 KB flash).

4.3 TinyML-Ready Model and Advanced Components

As a four-person team, the rubric expects at least three advanced components. We implemented three: (1) post-training INT8 quantization, (2) a multiple-model comparison, and (3) BLE communication of inference results. Each is described below.

Component 1 — INT8 post-training quantization. The deployed model is fully INT8 quantized. Edge Impulse performs this conversion for the on-device build, and Ananya’s notebook documents the same pipeline step by step: it loads the trained model, builds a representative dataset of 200 windows to calibrate the activation ranges, forces full-integer input and output for TFLite Micro, and exports a TFLite model plus a C header. Quantization reduced the model flash from 41.1 KB in float32 to 22.0 KB in INT8 — about a 46% reduction — while accuracy and latency stayed essentially unchanged. On top of this, the Edge Impulse EON Compiler reports a further 51% RAM and 48% ROM saving over a standard TFLite Micro build. The size and accuracy comparison appears in Section 6.

Component 2 — Multiple-model comparison. We trained two network sizes and compared them experimentally on accuracy and on-device cost. The deployed model is the 64–32–16 network at 92.7% and 22.0 KB. We also trained a smaller single-layer network (one dense layer of 16 units), which shrank the classifier flash to 15.2 KB but dropped validation accuracy to 84.8%, with the WATER class falling from 66.7% to 53.3%. Latency and RAM were essentially identical. The smaller model saves roughly 7 KB but loses nearly eight accuracy points, so we deployed the larger network. The two confusion matrices are shown in Section 6.

Component 3 — BLE communication of results. Inference output does not stop at the Serial Monitor. Each prediction is written to a BLE characteristic as a JSON string and consumed by a Web Bluetooth application that shows the live gesture and confidence, builds a sentence, and speaks the word using the browser’s text-to-speech. This is detailed in Section 5.

4.4 Why the Deployed Model Was Selected

The deployed 64–32–16 INT8 model was chosen because it met every success criterion with margin and won the head-to-head comparison. It reaches 92.7% accuracy at 22.0 KB of flash and 5.6 KB of RAM — far inside the 200 KB and 250 KB budgets — with a 63 ms single-inference estimate. The smaller alternative was rejected: its ~7 KB saving was irrelevant given how much budget remained, and it gave up nearly eight points of accuracy, hurting the already-weak WATER class most. Full integer quantization was preferred over the float32 build because it halves the flash for no measurable accuracy or latency cost.

5. TinyML System Design and On-Device Deployment

5.1 System Diagram and Pipeline

The system runs from sensor to speech. The IMU is sampled at 100 Hz; samples fill a sliding 2000 ms window buffer; when a full window is ready the INT8 classifier runs on the board; a confidence threshold on the softmax output decides whether to emit a class or UNKNOWN; the result is packed as JSON and written to a BLE characteristic; and a Web Bluetooth app receives it, displays the gesture, confidence, and latency, appends it to a sentence, and speaks it.

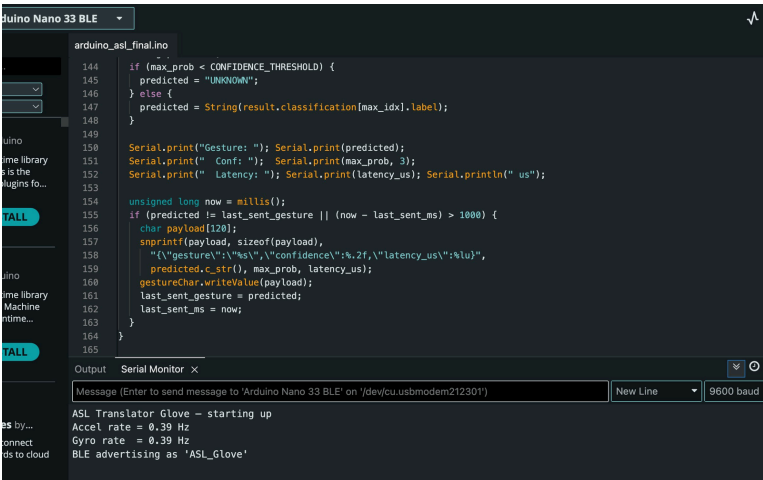
BMI270 IMU (6-axis @100 Hz) → 2000 ms sliding window → Spectral features → INT8 dense net (64-32-16) → softmax + 0.70 confidence threshold → BLE JSON notify → Web Bluetooth app: display + sentence builder + TTS

Figure 4. End-to-end system pipeline. Everything from the IMU through the INT8 classifier runs on the Arduino’s Cortex-M4F; only the recognized word leaves the board, over BLE.

5.2 Conversion and Arduino Integration

The trained network is converted to a full-integer INT8 TFLite model and compiled into an Arduino library by Edge Impulse, using the EON Compiler. The sketch includes that library directly and runs inference through the Edge Impulse runtime, which is TensorFlow Lite for Microcontrollers underneath. The sketch reads the IMU through the

Arduino_BMI270_BMM150 driver (the Rev2 sensor) and uses ArduinoBLE for the radio. At startup it verifies that the model expects fusion/accelerometer input over six channels before it begins inference; the build fails fast otherwise.



```

144 if (max_prob < CONFIDENCE_THRESHOLD) {
145   predicted = "UNKNOWN";
146 } else {
147   predicted = String(result.classification[max_idx].label);
148 }
149
150 Serial.print("Gesture: "); Serial.print(predicted);
151 Serial.print(" Conf: "); Serial.print(max_prob, 3);
152 Serial.print(" Latency: "); Serial.print(latency_us); Serial.println(" us");
153
154 unsigned long now = millis();
155 if (predicted != last_sent_gesture || (now - last_sent_ms) > 1000) {
156   char payload[120];
157   snprintf(payload, sizeof(payload),
158            "{\"gesture\":\"%s\",\"confidence\":%.2f,\"latency_us\":%u}",
159            predicted.c_str(), max_prob, latency_us);
160   gestureChar.write(payload);
161   last_sent_gesture = predicted;
162   last_sent_ms = now;
163 }
164
165

```

Output Serial Monitor X

Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem212301')

New Line 9600 baud

ASL Translator Glove - starting up
 Accel rate = 0.39 Hz
 Gyro rate = 0.39 Hz
 BLE advertising as 'ASL_Glove'

Figure 5. Inference and output logic in the Arduino sketch: the 0.70 confidence threshold producing UNKNOWN, the Serial print of gesture/confidence/latency, and the BLE JSON payload construction.

5.3 Deployment Constraints

Resource usage on the Arduino Nano 33 BLE Sense Rev2 (Cortex-M4F, 64 MHz), from the Edge Impulse EON Compiler estimate:

Resource	Value (INT8)	Budget / Target
Model flash (ROM)	22.0 KB	< 200 KB
Peak RAM (tensor arena)	5.6 KB	< 250 KB
Single inference latency	63 ms	responsive demo

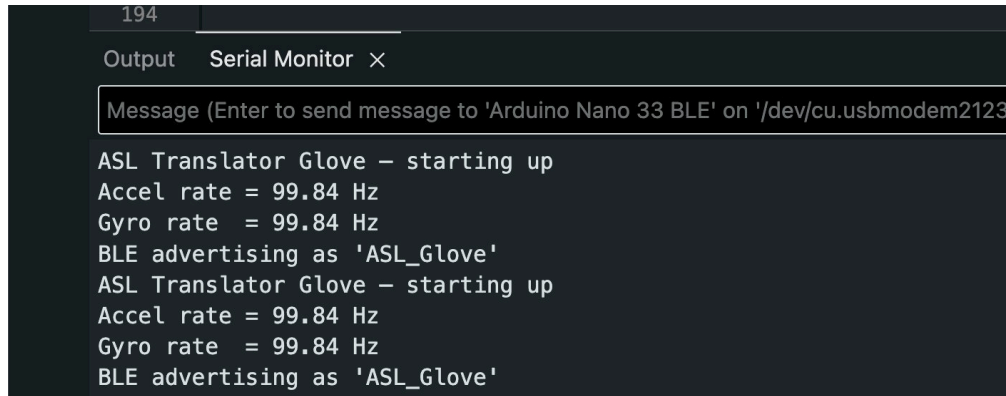
Table 2. On-device deployment constraints versus targets. All four success criteria are met with large margin.

Two latency numbers are worth separating. The 63 ms above is the time for one classifier pass, as estimated by the EON Compiler (62 ms spectral features + 1 ms classifier). The end-to-end latency the user perceives is longer — on the order of a few hundred milliseconds — because the board first fills a full 2000 ms window before each inference, and the serial logs show per-cycle times around 330 ms. The model itself is fast; the perceived delay is dominated by the window length, which is a design choice rather than a compute limit.

5.4 On-Device Inference Evidence

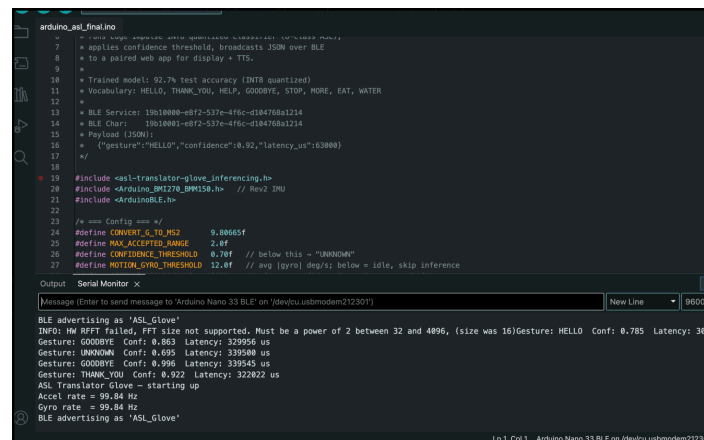
The board logs each prediction to the Serial Monitor and simultaneously streams it over BLE.

Figure references below show the device booting at the correct 100 Hz rate, the live web display connected over BLE, and the per-inference confidence and latency.



```
194
Output Serial Monitor X
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem212301')
ASL Translator Glove – starting up
Accel rate = 99.84 Hz
Gyro rate = 99.84 Hz
BLE advertising as 'ASL_Glove'
ASL Translator Glove – starting up
Accel rate = 99.84 Hz
Gyro rate = 99.84 Hz
BLE advertising as 'ASL_Glove'
```

Figure 6. Serial Monitor at startup: the IMU initializes at 99.84 Hz on both accelerometer and gyroscope, and the board advertises over BLE as “ASL_Glove.”



```
arduino_asl_final.ino
// ... (code snippets) ...
// Trained model: 92.7% test accuracy (INT8 quantized)
// Vocabulary: HELLO, THANK_YOU, HELP, GOODBYE, STOP, MORE, EAT, WATER
// BLE Service: 19b10000-e8f2-537e-4f6c-d104768a1214
// BLE Char: 19b10001-e8f2-537e-4f6c-d104768a1214
// Payload (JSON)
// { "gesture": "HELLO", "confidence": 0.92, "latency_us": 63000 }
// ... (code snippets) ...

#include <asl-translator-glove_inferencing.h>
#include <Arduino_RMT270_RPM158.h> // Rev2 IMU
#include <ArduinoBLE.h>

// ... (code snippets) ...

// #define CONFIDENCE_THRESHOLD 0.70f // below this = "UNKNOWN"
// #define MOTION_GYRO_THRESHOLD 12.0f // avg [gyro] deg/s; below = idle, skip inference

Output Serial Monitor X
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem212301')
New Line 9600
BLE advertising as 'ASL_Glove'
INFO: HW RFFT failed, FFT size not supported. Must be a power of 2 between 32 and 4096, (size was 16)
Gesture: HELLO Conf: 0.863 Latency: 329956 us
Gesture: UNKNOWN Conf: 0.695 Latency: 339540 us
Gesture: GOODBYE Conf: 0.996 Latency: 339545 us
Gesture: THANK_YOU Conf: 0.922 Latency: 322822 us
ASL Translator Glove – starting up
Accel rate = 99.84 Hz
Gyro rate = 99.84 Hz
BLE advertising as 'ASL_Glove'
```

Figure 7. Serial Monitor showing live on-device predictions with confidence and per-inference latency. Replace with a capture of ten live signs for the evidence table in Section 6.

5.5 BLE Data Format and Display

On each prediction the board writes a JSON string to a single BLE characteristic. The service UUID is 19b10000-e8f2-537e-4f6c-d104768a1214 and the characteristic UUID is 19b10001-e8f2-537e-4f6c-d104768a1214. The payload is:

```
{"gesture": "HELLO", "confidence": 0.92, "latency_us": 63000}
```

The web app filters by the service UUID rather than the device name, because the local name does not reliably appear in the advertising packet; it subscribes to the characteristic, parses each JSON notification, converts confidence to a percentage and latency to milliseconds, updates the

display, appends the word to a running sentence, and speaks it via the browser’s built-in text-to-speech. The 0.70 confidence threshold is applied on the board, so anything below it is shown as UNKNOWN and never spoken. No server or external API is involved.

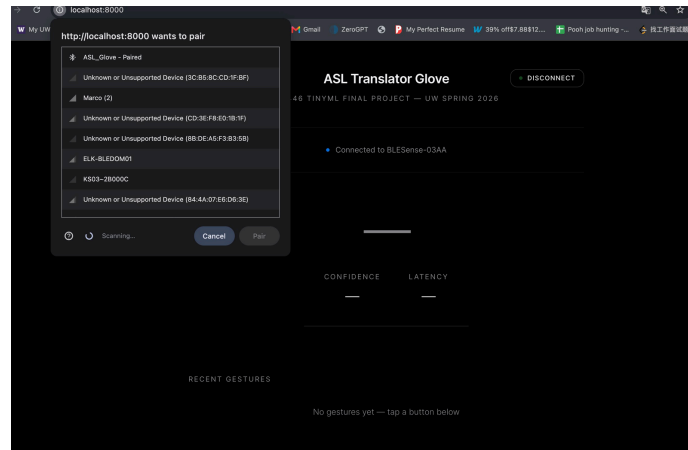


Figure 8. Web Bluetooth application paired with the board (“Connected to BLESense-03AA”), with confidence, latency, recent-gestures, and sentence areas. The app runs entirely in Chrome with no server.

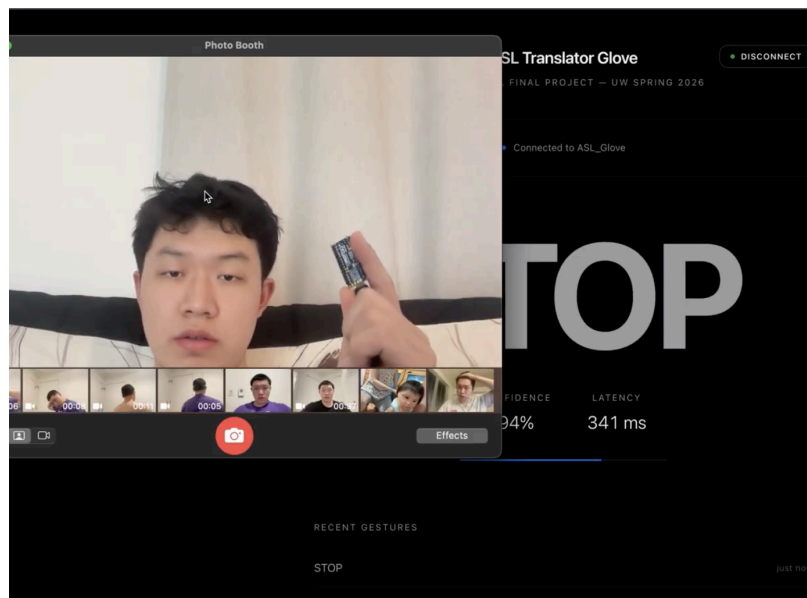


Figure 9. The wearable in use: the Arduino Nano 33 BLE Sense Rev2 worn on the hand while performing a sign.

6. Results and Analysis

6.1 Metric Selection and Justification

Task type	Metrics used and justification
8-class classification (mild imbalance, 75–101 per class)	Overall accuracy, per-class accuracy, F1 per class, weighted precision/recall/F1, AUC, and the full confusion matrix. With mild class imbalance, overall accuracy alone can hide a weak class, so the confusion matrix and per-class F1 are reported to

Task type	Metrics used and justification
	show exactly which signs are confused — which matters more for a usable device than a single headline number.

Table 3. Metric selection and justification for the classification task.

6.2 Overall Results

The deployed INT8 model reaches 92.7% validation accuracy, with an AUC of 0.99 and weighted precision, recall, and F1 all at 0.93. Most signs are recognized reliably; GOODBYE and THANK_YOU reach 100% and five others are at or above 93%. The clear weak spot is WATER at 66.7%, most often misread as HELLO (26.7%). We attribute this to a sensor limit rather than a training problem: the two gestures share a similar overall hand motion that a six-axis IMU cannot fully separate.



Figure 10. Confusion matrix and per-class F1 for the deployed INT8 model (validation set). Diagonal entries are per-class accuracy; WATER's errors fall into HELLO (26.7%) and HELP (6.7%).

Per-class accuracy and most-confused class (deployed INT8 model):

Class	Accuracy	F1	Most confused with
EAT	94.7%	0.97	MORE (5.3%)
GOODBYE	100%	0.98	—

Class	Accuracy	F1	Most confused with
HELLO	94.4%	0.87	STOP (5.6%)
HELP	93.8%	0.94	MORE (6.3%)
MORE	90.9%	0.91	GOODBYE/STOP (4.5%)
STOP	95.5%	0.93	WATER (4.5%)
THANK_YOU	100%	1.00	—
WATER	66.7%	0.77	HELLO (26.7%)

Table 4. Per-class accuracy, F1, and dominant confusion for the deployed model. Weighted P/R/F1 = 0.93; AUC = 0.99.

6.3 Big vs. TinyML Model and the Quantization/Size Comparison

Model	Val. accuracy	Flash	RAM / Latency	Selected?
Dense 64-32-16, float32	92.7%	41.1 KB	5.6 KB / 64 ms	No — larger flash
Dense 64-32-16, INT8	92.7%	22.0 KB	5.6 KB / 63 ms	Yes — deployed
Dense 16 (smaller), INT8	84.8%	15.2 KB	5.6 KB / 63 ms	No — ~8 pts acc.

Table 5. Model comparison across precision and architecture. INT8 quantization halves flash for no accuracy loss; the smaller network saves ~7 KB but loses ~8 accuracy points.

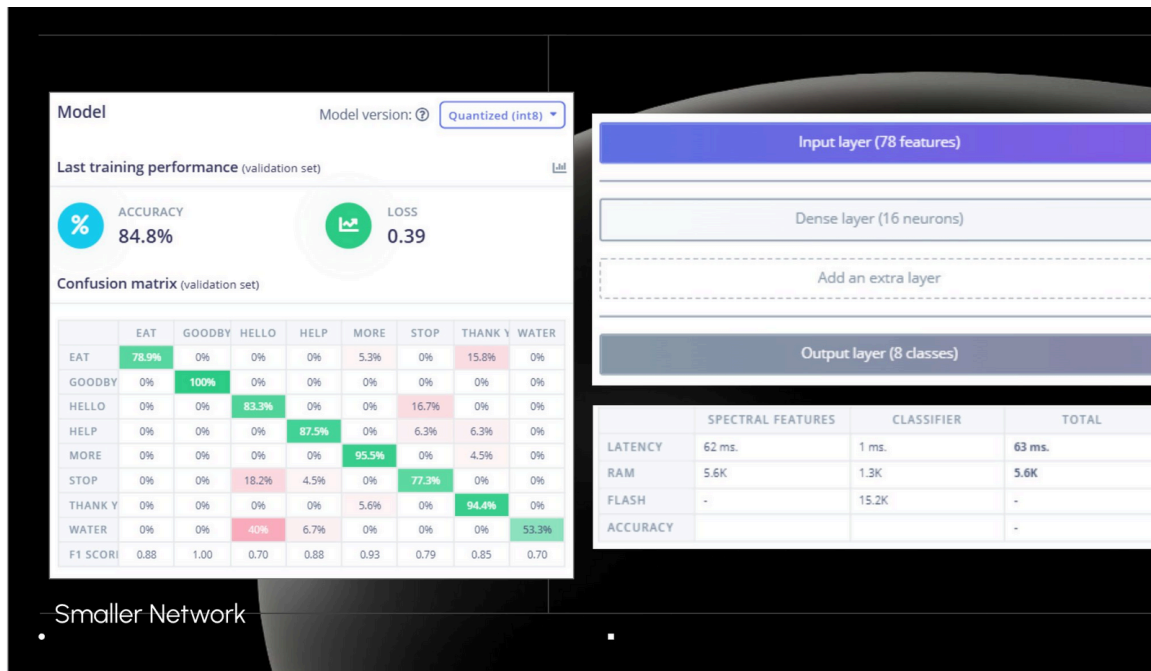


Figure 11. Smaller single-layer network (one dense layer of 16 units) in Edge Impulse: 84.8% accuracy and 15.2 KB classifier flash. WATER falls to 53.3%, confirming the larger network was the right deployment choice.

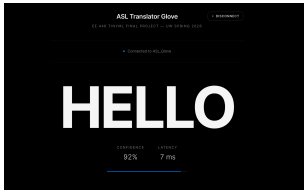
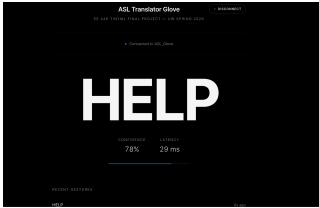
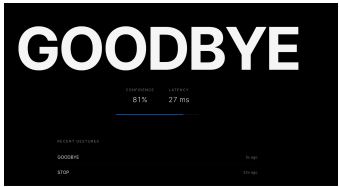
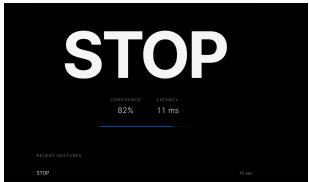
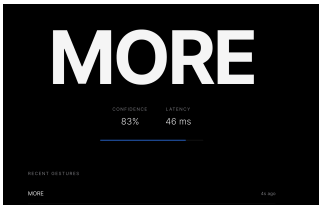
6.4 Deployment Resource and Performance Summary

Hardware	Model / Flash	RAM / Arena	Output evidence
Arduino Nano 33 BLE Sense Rev2	INT8 / 22.0 KB	5.6 KB	Serial Monitor + BLE + Web app

Table 6. Deployment resource and performance summary for the target hardware.

6.5 On-Device Inference Evidence (10 Independent Instances)

The table below records ten independent live signs performed on the board. Fill the predicted column and correctness from the captured run, and point each row at the corresponding web-app screenshot.

ID	Input (sign)	Predicted	Correct?	Evidence
1	HELLO	HELLO	[Yes/No]	
2	THANK YOU	THANK YOU	[Yes/No]	
3	HELP	HELP	[Yes/No]	
4	GOODBYE	GOODBYE	[Yes/No]	
5	STOP	STOP	[Yes/No]	
6	MORE	MORE	[Yes/No]	

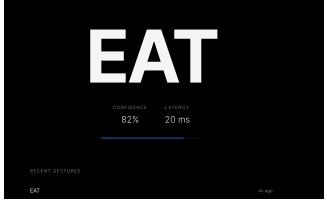
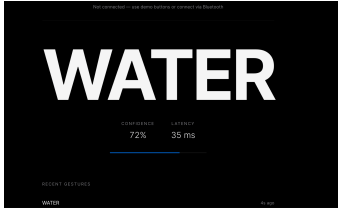
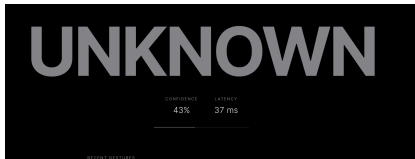
ID	Input (sign)	Predicted	Correct?	Evidence
7	EAT	EAT	[Yes/No]	
8	WATER	WATER	[Yes/No]	
9	Quickly Waving	Quickly Waving	[Yes/No]	
10	Resting	Resting	[Yes/No]	

Table 7. On-device inference evidence for ten independent instances. Record real results, including any misreads.

7. Challenges, Limitations, and Future Work

7.1 The Threading Bug

The toughest problem was on deployment. The first sketch ran inference in a background thread while the main loop filled the IMU buffer. On the Mbed scheduler the inference thread consumed nearly all of the CPU, so the main loop barely ran and the buffer stopped receiving new samples. The board then predicted the same sign at high confidence regardless of motion. A debug print of the raw IMU values showed they were frozen. The fix was to drop the background thread and use a single loop that fills one full window and then runs inference once. After that, the board produced distinct predictions for distinct motions. We also found and fixed an unsigned-integer underflow in the tick-timing code that could make a wait run for an extremely long time.

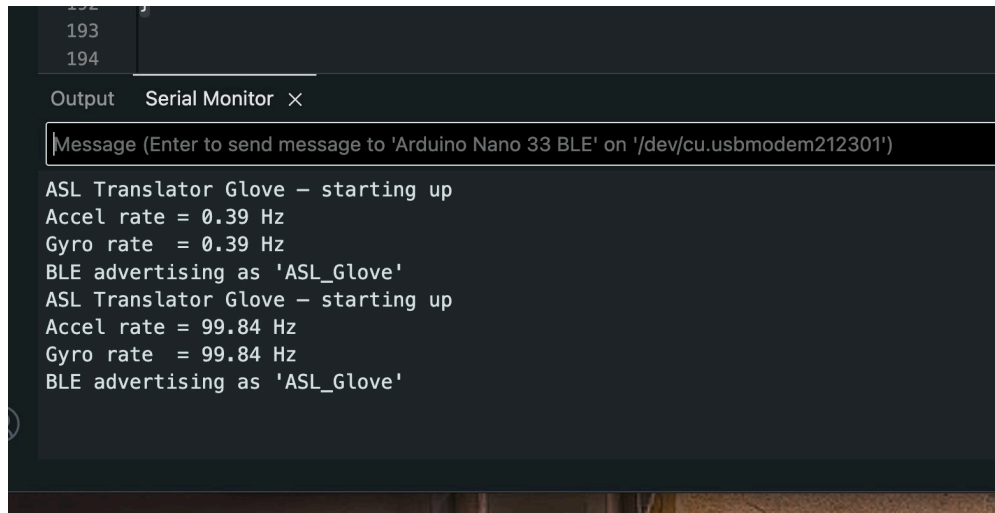


Figure 12. BMI270 startup quirk: on the first boot the IMU reports 0.39 Hz (top), too slow to recognize signs; after a reset it initializes correctly at 99.84 Hz (bottom). Documented as a known limitation.

7.2 What Worked, Partially Worked, and Did Not

Worked: the end-to-end pipeline from sensor to spoken word, the BLE link, and reliable recognition of the stronger signs (six of eight at $\geq 93\%$). **Partially worked:** WATER, which the model recognizes about two-thirds of the time and confuses with HELLO. **Did not work as first built:** the multi-threaded inference design, which had to be replaced with a single-threaded loop.

7.3 Limitations and Future Work

Limitations. The vocabulary is limited to eight motion-based signs because the IMU cannot sense handshape; WATER is the weak class; and inference runs once per full window, so there is a short delay between making a sign and seeing the result.

Future work. Two realistic next steps. First, add flex sensors on the fingers so the device can read handshape and cover signs the IMU cannot currently distinguish. Second, switch to overlapping sliding-window inference so the prediction updates continuously instead of once per window, which would reduce the delay the user feels.

8. Writing Quality, Reproducibility, and References

8.1 Reproducibility

The full implementation is public at github.com/Jason-lab1126/asl-translator-glove. The Arduino sketch is in `deployment/arduino_asl_final/`, the INT8 quantization reference notebook in `compression/`, the web app in `webapp/`, and the model-to-deployment contract (input shape, class

order, BLE schema, confidence threshold) in INTERFACE.md. The trained model, dataset, and impulse design are in the public Edge Impulse project (studio.edgeimpulse.com/public/1022005). Tools and libraries: Edge Impulse Studio for data, training, INT8 export, and the EON Compiler; TensorFlow Lite for Microcontrollers as the on-device runtime; Arduino IDE 2.3.8 with the Arduino_BMI270_BMM150 and ArduinoBLE libraries; and the Web Bluetooth API for the browser display. Hardware: Arduino Nano 33 BLE Sense Rev2.

8.2 References

- [1] Edge Impulse Inc. Edge Impulse Documentation and Studio. <https://docs.edgeimpulse.com> (accessed June 2026). Project ID: 1022005.
- [2] Warden, P. & Situnayake, D. TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media, 2020.
- [3] Arduino. Nano 33 BLE Sense Rev2 Datasheet. <https://docs.arduino.cc/hardware/nano-33-ble-sense-rev2> (accessed June 2026).
- [4] Bosch Sensortec. BMI270 Inertial Measurement Unit Datasheet. Bosch Sensortec GmbH, 2021.
- [5] TensorFlow Authors. TensorFlow Lite for Microcontrollers. <https://www.tensorflow.org/lite/microcontrollers> (accessed June 2026).
- [6] W3C Web Bluetooth Community Group. Web Bluetooth API Specification. <https://webbluetoothcg.github.io/web-bluetooth/> (accessed June 2026).
- [7] World Federation of the Deaf. Information on Sign Languages. <https://wfd.fi> (accessed June 2026).